



OctopusTaint: Advanced Data Flow Analysis for Detecting Taint-Based Vulnerabilities in IoT/IoT Firmware

Abdullah Qasem*
Security Research Centre
Concordia University
Montreal, Canada

Mourad Debbabi
Security Research Centre
Concordia University
Montreal, Canada

Andrei Soeanu
Security Research Centre
Concordia University
Montreal, Canada

Abstract

The widespread integration of Internet of Things (IoT) and Industrial IoT (IIoT) devices in respectively home and business environments offers both benefits and perils. While these devices, such as IP cameras and network routers improve operational efficiency with their user-friendly web interfaces, they also broaden the potential for cybersecurity vulnerabilities. Recent studies highlight the vulnerability of these devices to taint-based attacks, demonstrating that even attackers with limited permissions can gain control of a device. Current *state-of-the-art* solutions for mitigating these risks primarily utilize Dynamic Symbolic Execution (DSE). Although effective, DSE is computationally costly and challenging for large-scale analysis. Besides, during inspection, these approaches typically exhibit over-taint behavior by producing a large number of alerts, many of which are false positives due to ineffective handling of sanitization measures that might be in place.

To overcome these limitations, we introduce *OctopusTaint*, an innovative static-based taint analysis approach that integrates advanced data flow analysis with backtracking techniques. *OctopusTaint* is distinguished by its integration of a sanitization inspection module and sophisticated post-processing filters. These features are specifically designed to minimize false positives effectively while ensuring the accurate identification of genuine security threats. *OctopusTaint* also excels in tracking transformed tainted inputs across NVRAM, identifying new user-defined taint source functions while addressing the challenges associated with indirect calls and aliasing. Through comparative performance evaluations, *OctopusTaint* demonstrates superior performance over the current *state-of-the-art* solutions, *SaTC*, *EmTaint*, and *MangoDFA*. It reports genuine extra tainted sinks in considerable less time (24% faster). Furthermore, *OctopusTaint* identifies 82% of tainted sinks within *EmTaint*'s labeled dataset while exhibiting its advanced capability in sanitization inspection. It correctly flags as sanitized 320 sinks, which were misidentified as genuine alerts by *EmTaint*. Furthermore, *OctopusTaint* uncovers additional candidates overlooked by *EmTaint*, leveraging its enhanced detection mechanisms for new taint sources. *OctopusTaint* successfully identifies 142 *n*-day vulnerabilities previously reported by *SaTC* and *EmTaint*, in addition to discovering dozens of potential 0-day candidates.

*Corresponding author: a_qas@encs.concordia.ca



This work is licensed under a Creative Commons Attribution International 4.0 License.

CCS '24, October 14–18, 2024, Salt Lake City, UT, USA
© 2024 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-0636-3/24/10
<https://doi.org/10.1145/3658644.3690307>

CCS Concepts

• Security and privacy → Software reverse engineering.

Keywords

Vulnerability Detection, IoT and IIoT Security, Firmware Analysis, Static Analysis, Taint Analysis, Data Flow Analysis, Sanitization Inspection

ACM Reference Format:

Abdullah Qasem, Mourad Debbabi, and Andrei Soeanu. 2024. OctopusTaint: Advanced Data Flow Analysis for Detecting Taint-Based Vulnerabilities in IoT/IIoT Firmware. In *Proceedings of the 2024 ACM SIGSAC Conference on Computer and Communications Security (CCS '24)*, October 14–18, 2024, Salt Lake City, UT, USA. ACM, New York, NY, USA, 15 pages. <https://doi.org/10.1145/3658644.3690307>

1 Introduction

The proliferation of IoT technologies has led to a surge in the interest of malevolent actors who are actively searching for vulnerabilities within these ubiquitous devices. These devices, offering web interfaces for configuration, control, management, and data sharing, have become prime targets for cyber attacks. Recent studies have revealed that IoT devices with web interfaces are still prone to taint-based vulnerabilities [1, 2]. Moreover, in 2021 alone, approximately one billion IoT devices were compromised, and the routers are the most vulnerable among IoT devices, with 46% [3]. By compromising millions of devices, the attackers can transform them into powerful armies that amplify their capabilities providing them the means to conduct large-scale and debilitating attacks, reminiscent of the infamous Mirai attack [4]. Actually, such threats can be significantly more dangerous when vulnerable IIoT devices are deployed in Industrial Control Systems (ICS). In order to forestall the occurrence of such destructive incidents, there is an urgent need for innovative solutions that can effectively mitigate the inherent risks associated with IoT/IIoT devices.

Recently, several methodologies have been advanced to detect taint-based vulnerabilities in Linux-based IoT devices. These include command injection and buffer overflow vulnerabilities within the firmware. Prominent among these are, to name a few, *Karonte* [5], *SaTC* [6], and *EmTaint* [7]. These proposed solutions, combining static analysis with DSE, have demonstrated enhanced suitability and robustness in detecting vulnerabilities in embedded firmware systems. However, these solutions have the following limitations: First, their efficacy is entirely dependent on the application of DSE as a taint engine. The reliance on DSE as a primary taint analysis engine necessitates the identification of all potential execution paths within the Control Flow Graph (CFG), particularly those

paths where sensitive sinks (e.g., `system`) are influenced by values returned from source functions (e.g., `get_cgi`) receiving user-controlled inputs. This methodology demands a thorough symbolic exploration to confirm that all relevant paths are assessed, which can result in prohibitive computational costs due to state or path explosions, or require an impractical amount of time to complete. Moreover, these approaches track tainted inputs from Source to Sink. However, in certain scenarios, an analysis initiated from the sinks (e.g., `system`) and tracing backward to the sources (e.g., `get_cgi`) can be faster [8]. In response to the identified limitations, we introduce *OctopusTaint*, an entirely static analysis technique. *OctopusTaint* advances the Reach Definition Analysis (RDA) methodology by incorporating enhancements through the *clinic* module of the *angr* framework. This approach utilizes *use-def* and *def-use* chains, structuring the analysis around a Data Dependency Graph (DDG), which allows *OctopusTaint* to comprehensively cover all statements in the targeted function without being affected by the state explosion problem. Additionally, *OctopusTaint* optimizes the analysis by initiating the process from the Sink and tracing back to the Source, focusing exclusively on paths pertinent to the Sink. This targeted analysis significantly accelerates the process, enabling *OctopusTaint* to perform tasks in minutes that would typically take an hour with DSE, thereby positioning it as a faster alternative to contemporary *state-of-the-art* techniques.

Second, a crucial limitation of the existing solutions is that they do not take into account sanitization measures, which might be implemented by developers to validate the format of the inputs or to prevent misuse. This oversight often results in a deluge of alerts, many of which are false positives, stemming from inadequate inspection of these sanitization measures. To overcome this challenge, in our solution, we propose and implement a novel strategy that factors the efficacy of sanitization code within the paths analyzed, inspired by identifying missing security checks in Linux kernel [9]. This pivotal process is essential for distinguishing between secure and potentially hazardous tainted paths. By systematically identifying and validating the presence of developer-implemented sanitization measures, our methodology proficiently differentiates between genuinely risky paths and those safeguarded through proper input handling. This enhanced level of analysis significantly elevates the precision of our security evaluations and substantially reduces the rate of false positives. Our approach is detailed in Section 3, Section 4.8, and Section 5.4, which cover Static Taint Analysis, Sanitization Inspection, and Post-Processing Filtering Effectiveness, respectively, detailing how each component contributes to the overall performance and robustness of our security framework.

The third shortcoming of the existing security solutions is their reliance on exact keyword matches between the front-end (web interface files) and the back-end (binary files). However, our research reveals that keywords identified within the web interface may undergo transformations when stored in shared memory using NVRAM, and subsequently retrieved under a different identifier in the same or in a different binary file. This process results in indirect, transitive vulnerabilities that remain undetectable by current methodologies, such as *SaTC*. For example, the vulnerability detailed in Section 2.2 and demonstrated in Listing 2 is not identified by existing tools due to these keyword transformations. To overcome this limitation, we have developed a recursive tracking

method to trace and correlate related keywords, thereby offering a more thorough vulnerability detection mechanism. The specific details of this method are provided in Section 3 and Section 4.9.

The fourth critical gap in the existing solutions is that they do not consider user-defined functions that engage with specified keywords and assign related values to memory buffers, such as stack variables or global memory variables. These functions, analogous to user input receiver functions like `get_cgi`, are pervasive throughout binary programs, and their exclusion from security analyses leads to undetected vulnerabilities, a gap not addressed by the current *state-of-the-art* approaches, such as *SaTC* [6], *Karonte* [5], and *EmTaint* [7]. Our research has thoroughly investigated this limitation, highlighting its critical impact on the security framework. In order to bridge this gap, our approach includes an exhaustive inspection of these user-defined functions, which allows to uncover vulnerabilities previously not noticed. Detailed discussions of our methodological innovations are presented in Section 4.6.2 and Section 5.5.

In essence, the main contributions of this work are as follows:

- We propose *OctopusTaint*, a novel data flow analysis approach to tackle the problem of taint-based vulnerabilities in IoT and IIoT firmware images that employ web interfaces for management.
- We design and implement an inspection strategy to detect the existence of sanitization code in the inspected paths in order to prune secure tainted paths.
- We design and implement a method to connect tainted execution paths, that share information through NVRAM over functions in the same binary or across different binary files within the same firmware.
- We design and implement a procedure to identify new user-defined functions that receive user inputs based on given keywords.
- We evaluate the performance of *OctopusTaint* on real-world firmware. Our results demonstrate that *OctopusTaint* outperforms the *state-of-the-art* approaches, i.e., *EmTaint* [7], *SaTC* [6], and *MangoDFA* [10], and is able to discover a significant number of undisclosed high-risk vulnerabilities.

We upload the source code, dataset, and experiment results to this repository¹ for evaluation.

2 Background

2.1 Threat Model

Our research is focused on detecting taint-based vulnerabilities within the firmware of IoT/IIoT devices, particularly those used either in home or business environments, such as routers, cameras, or IIoT devices used in ICS - including power grid devices, such as PMUs, Protection relays, PDCs, etc. These IoT/IIoT devices typically provide web interfaces for configuration, control, management, and data sharing. In our threat model, we consider a scenario where the attackers have access to the same or similar firmware of a target device. They can acquire the device model from the vendor in order to craft and exercise their attacks. Furthermore, they can access the web interface of the target devices via a communication network.

¹<https://bit.ly/4aVmTNq>

This attack model reflects a realistic and increasingly common threat landscape, where malevolent actors can exploit software vulnerabilities remotely (without the need to physically access the target devices of their potential victims). The aim is to identify and address these vulnerabilities, thereby strengthening the security of IoT and IloT devices against such remote attacks.

2.2 Motivating Examples

Our research reveals that IoT firmware exposing web interfaces exhibit taint-based vulnerabilities that are susceptible to two types of exploits, relying on direct and respectively indirect vulnerabilities. The first type involves direct taint propagation where a user-input receiver function (e.g., `get_cgi`), obtains input data from a user-controlled input field in the web interface. This input data may subsequently be subjected to various string manipulation functions (e.g., `strcpy`, `sprintf`). Ultimately, the manipulated data finds its way into a system call function (e.g., `system`), which executes commands at the operating system level. An illustrative example of this direct vulnerability type is outlined in Listing 1 taken from the `httpd` binary file within `linksys_e1200(v2.0.11)`. Specifically, the `get_cgi` function receives user input data at line 3, data which is subsequently passed to the `snprintf` function as a parameter at line 5 in order to format the string. Finally, this formatted string is utilized as an argument in the `system` call at line 7, where the potential for exploitation is apparent.

Listing 1: Direct Taint-based Vulnerability

```
1 void hnd_parentalctrl_unlock(){
2     ..
3     puVar3= get_cgi("HND_block_mac");
4     ..
5     snprintf(acStack555,0x200,
6         "echo \"*+/%s\"> proc/unblock_proc", puVar3);
7     system(acStack555)
8     ..
9 }
```

The second type of vulnerabilities can be exploited by following a two-step process. Initially, a function (e.g., `get_cgi`), obtains user-supplied input data. Then, this input data undergoes processing through various string manipulation functions before being committed to a shared memory space, exemplified by `nvrw_set`. Subsequently, another function, either within the same or a different binary file in the same firmware, retrieves this data from the shared memory using a function such as `nvrw_get`. This other function then processes the data further through additional string manipulations. Similar to the first vulnerability type, this concludes with the processed data being used in a `system` call function (e.g., `system`), which allows to execute commands at the operating system level. Thus, such vulnerabilities are classified as indirect taint-based vulnerabilities. The indirect vulnerability type can also manifest in a transitive mode, which involves the keyword-value storing feature of the shared memory whereby user input, originally associated with a particular keyword, is stored under a different keyword, thereby complicating traceability. Listing 2 demonstrates this intricate vulnerability pathway within the `httpd` binary of the Netgear `r6400` firmware. The process begins with the `FUN_184d4` function, which collects input using the keyword "ptp_mac_an" at line 3, and then stores it in NVRAM under a different keyword "wlg_wds_ptp_mac" at line 5. Subsequently, within

the `FUN_5f718` function, this re-keyed input is retrieved at line 9 and processed through `strcpy` at line 10. This tainted input is then inter-procedurally transferred and further manipulated through `sprintf` at line 16, culminating in its use within a `system` call at line 17. This processing sequence illustrates how such vulnerabilities can be exploited in multiple steps, with data being changed and moved around in a way that masks where it originally come from and how it is altered.

Listing 2: In-direct and Transitive Vulnerability

```
1 void FUN_5fc60(){
2     ..
3     FUN_184d4(param1, "ptp_mac_an", acStack32);
4     ..
5     acosNvramConfig_set("wlg_wds_ptp_mac", acStack32);
6 }
7 void FUN_0005f718(){
8     ..
9     Var2 = acosNvramConfig_get("wlg_wds_ptp_mac");
10    strcpy((char*)&local_120, Var2);
11    iVar = FUN_5f564(local_120, &DAT_001a3914);
12    ..
13 }
14 void FUN_5f564(param_1, param_2){
15     ..
16    sprintf(acStack108, "rm -f %s;", param_1);
17    system(acStack108);
18 }
```

2.3 Challenges

Designing and implementing a static taint engine is a complex problem. In this section, we outline the major challenges that we had to overcome in order to achieve our goal.

- **Stack Variables and Global Variables Identification:** Precisely identifying the requisite stack variable memory offset necessary for creating, renewing, or retrieving stack memory definitions poses notable challenges, particularly when indirect memory addressing or copy propagation is involved. For example, RDA over *VEX-IR* occasionally interprets register operations utilized to calculate memory reference addresses as mathematical operations, hindering the update of the requisite memory definitions.
- **Indirect Call Handling:** Indirect function calls are prevalent across the inspected firmware and are commonly associated with potentially vulnerable paths [7]. Determining the appropriate function for exploration in the existing RDA implementation of the *anqr* framework, in the presence of indirect call, requires guidance, in the absence of which crucial tainted paths may be missed.
- **Recover Functions Parameters:** Throughout our analysis, we frequently observe that the *anqr* framework was unable to completely recover all expected parameters. This limitation of the *anqr* framework often leads to the unintentional exclusion of important tainted variables that are essential for the complete taint analysis.
- **Sanitization Inspection:** Identifying security checks and length constraints on tainted paths within the Data Dependency Graph (DDG), generated during RDA, presents substantial challenges. During the backtracking phase, the process navigates the DDG to pinpoint all definitions that contribute to the final sink of a tainted path. However, these definitions are not always consistently applied across all

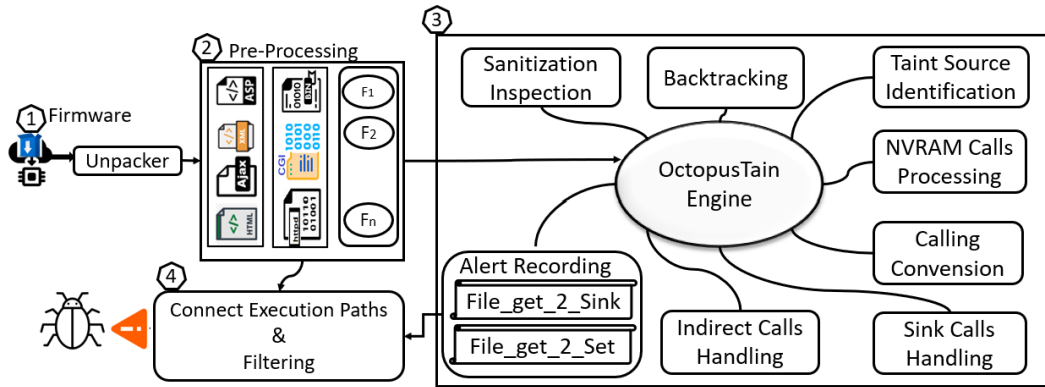


Figure 1: *OctopusTaint* Architecture and Workflow

basic blocks along the path from the source to the sink. For example, a definition that might originate in the source's basic block, can be used thereafter in a string manipulation function (e.g., `sprintf`) within a subsequent block, disappear across several intervening blocks, and then reappear in the sink function at the terminal block. This discontinuous utilization poses a significant challenge, as relying solely on the DDG may result in missing numerous basic blocks that potentially perform critical security checks on the definitions leading eventually to the sink. Our analysis reveals that these frequently overlooked blocks often contain essential security checks, which do not generate new definitions or create additional edges in the DDG. As a result, many alerts reported could be in fact false positives.

3 OctopusTaint Workflow

In this section, we detail the architecture and workflow of *OctopusTaint*, our innovative approach for detecting taint-based vulnerabilities in IoT firmware image. Figure 1 illustrates this workflow, which involves four pivotal stages. The first stage is firmware unpacking. The pre-processing stage selects relevant front-end and back-end files through keyword matching, along with the selection of binary functions. At the core of our system is the elaborated *OctopusTaint* engine, that integrates Reaching Definition Analysis with clinic module capabilities and DDG. Also, this engine coordinates eight components: Sanitization Inspection, Backtracking, Taint Source Identification, Libc Functions Emulation, Calling Convention Creation and Recovery, NVRAM Calls Processing, Indirect Call Handling, and Alert Recording. The final stage involves Connect Execution Paths based on recorded information and implementing Filtering Procedures to refine the analysis, isolating the most critical vulnerabilities from the broader set of recorded alerts.

Firmware Unpacking: We use *binwalk* [11] to unpack the firmware to make its internal components accessible for analysis.

Back-end Files Selection: We use *SaTC* [6] procedures to extract string keywords from various web interface files including *HTML*, *XML*, *Ajax*, and *ASP*. Then, binary files that exhibit an overlap or similarity in terms of keywords are selected for next-stage

analysis. Additionally, we have enhanced the *SaTC* procedure to specifically target and select binary files that utilize shared memory library, such as "`libnvr.so`", as a method for facilitating information exchange across binary files.

Binary Functions Selection: We prioritize binary functions that have *caltees* matching the following criteria: (1) responsible for receiving user input (e.g., `get_cgi`), (2) executing system commands (e.g., `system`), and (3) performing NVRAM operations (e.g., `nvr_set`, `nvr_get`). By selecting binary functions based on these criteria, our approach ensures that our analysis focuses on firmware areas that are most likely to contain vulnerabilities. Besides, while iterating over all selected functions, we gather a list of newly identified user-defined functions designed to receive user input, as detailed in Section 4.6.2. Equipped with this data, we then examine binary functions that make use of these functions while not being part of the previous analyses.

Static Taint-Engine and Backtracking: *OctopusTaint* performs Reaching Definition Analysis in both intra-procedural and inter-procedural modes. It is built atop *use-def* chains and *def-use* chains and represented within a DDG. It employs RDA over *VEX-IR* to statically generate and determine which definitions can propagate to specific code points. Further enhancing its functionality, *OctopusTaint* integrates the *AIL* intermediate language, a component of the *clinic* module, allowing to achieve more precise recovery of call conventions and stack and global memory variables. This enhancement is crucial for the generation and propagation of definitions during the emulation of required functions. Given a selected function, we pass it through our implemented taint-engine to create its related reaching definitions. Upon encountering a system call (e.g., `system`) or a dangerous memory writing function (e.g., `strcpy`, `sprintf`), it carefully examines the inspected passed parameters' definitions. The process involves a comprehensive backward traversal employing a bottom-up depth-first approach, starting from the sink and traversing the DDG generated by the RDA. If a connection with the taint source definition is established, *OctopusTaint* records this path in a dedicated file (`File_get_2_Sink`) pointing out the Source and the Sink details. Additionally, the taint-engine extends to handling NVRAM transitive operations (e.g., `nvr_set`). In these instances, the same rigorous approach is applied, with the resulting

paths logged in a separate file (`File_get_2_Set`) for subsequent post-processing.

Sanitization Inspection: We thoroughly examine candidate paths resulting from the backtracking procedure, identifying key security checks, notably conditions, that might affect the definition of related sinks. *OctopusTaint* focuses on extracting such checks, as they are pivotal in determining if a tainted sink is subjected to input validation or protective measures against malicious data. Special attention is given to string manipulation functions and their length parameters encountered along these paths. This information, along with the backtracking details, is recorded in either the `File_get_2_Set` or `File_get_2_Sink`. By leveraging the obtained data, we then eliminate tainted paths that pass through robust security checks or have restricted input lengths, effectively minimizing false positives and ensuring that reported paths are genuinely free of sanitization.

Connect Execution Paths and Filtering: Firmware systems involve compositions of multiple binary files, with the nature of specific tasks often necessitating interaction and communication among functions residing within a single binary or spanning across multiple binaries. Firmware designers employ various interaction mechanisms, such as shared memory, sockets, or files, to establish communication. We focus on tracing communication pathways through shared memory over NVRAM. Building upon the previously detailed stages, each analyzed binary file now has its corresponding `File_get_2_Set` and `File_get_2_Sink` files.

4 OctopusTaint Implementation

In this section, we elaborate on the internal design and implementation of the components of *OctopusTaint*. We developed the *OctopusTaint* using approximately 7,722 lines of Python code. *OctopusTaint* utilizes Reaching Definition Analysis, Data Dependency Graph, and the clinic module built-in the *angr* framework (v9.2.44) [12].

4.1 Static Taint-Engine

OctopusTaint extends the implementation of Reach Definition Analysis (RDA) over *VEX-IR* intermediate representation at the function level, coupled with the clinic module, which operates over the *AIL* (Angr Intermediate Language) for function emulation during RDA generation. *OctopusTaint* employs the *AIL* representation to accurately simulate the effects of system calls for string manipulations, such as `strcpy` and `sprintf`, in order to ensure that the analysis state is consistently updated to reflect the true behavior of these functions. While both *VEX-IR* and *AIL* formats share substantial similarities, *AIL* provides distinct advantages in handling stack variables directly, whereas *VEX-IR* offers only stack addresses. This capability makes *AIL* the preferred choice for *OctopusTaint*, simplifying the handling of challenging variables. In practice, *OctopusTaint* leverages the clinic module within *angr* to transform pertinent *VEX-IR* blocks into *AIL*. The clinic module brings to the table robust features such as variable recovery analysis for identifying variables and copy propagation to eliminate redundant data, enhancing efficiency and accuracy in vulnerability analysis.

4.2 Selective Inter-Procedural Analysis

Inter-procedural reaching definitions analysis is not part of the *angr* framework (v9.2.44). To make up for this, *angr* provides a handler that lets developers choose how to handle local functions. The strategy is to execute the analysis recursively. Whenever a call to a local function is detected, a new *ReachingDefinitionsAnalysis* child process is started on the target function. To preserve the continuity of the analysis, we pass to the child RDA the current state of its parent. When the child's RDA finishes, its end-state is copied back to the parent. This lets the parent analysis pick up where it left off right after the function call, as outlined in [13]. To optimize our implementation, we only expand our analysis if at least one of the parameters of the targeted function is tainted by a recognized taint source; otherwise, the function call is ignored, and the RDA generation continues from the subsequent statement.

4.3 Libc Functions Emulation

RDA presents a tailored interface for reverse engineering developers, facilitating the creation of customized handlers for each system call of interest. This provision allows developers to effectively address and manage individual system calls, according to their specific needs. Upon encountering a function call linked to a system call of interest during the reaching definition generation, the related handler is triggered to perform its corresponding effects. For instance, to accurately emulate the `strcpy` function during the generation of reaching Definitions, it is necessary to incorporate a `handle_strcpy` method within our custom RDA class. The `handle_strcpy` method is tasked with updating the memory definition of the first argument of the `strcpy` function encountered at the current instruction address. Subsequently, it adds a new edge to the RDA's DDG. This edge is created from the memory definition of the second parameter of `strcpy` to the updated memory definition of the first parameter of `strcpy`.

4.4 Recover Function Parameters

During RDA creation, sometimes the *angr* framework fails to retrieve some or all the parameters of called functions. This issue manifests in two forms. Firstly, the current RDA implementation, when operating on *VEX-IR*, sometimes fails to fully recover all function parameters. To circumvent this limitation, we utilize the parameters identified by the *clinic* module within its corresponding block. However, in some cases clinic module cannot fully recover all the necessary parameters. This was particularly evident with functions like `sprintf` and `scanf`, which tend to involve a diverse mix of parameters more than four parameters, including registers, constants, stack variables, or global variables, as part of their string formatting logic. To address these complexities, our approach involves a detailed inspection of the instructions preceding the function call. Leveraging our understanding of different architectures, specifically MIPS and ARM, we tailor our strategy accordingly. In MIPS architecture, parameters are typically passed using registers `a0` through `a3`, whereas in ARM architecture, they are passed through `r0` to `r3`. For scenarios involving more than four parameters, the passing occurs through a specific memory location. Utilizing this knowledge, we manage to recover missing parameters by examining the relevant instructions before the function call.

Algorithm 1 A bottom-up depth-first traversal over RDA DDG

Require: *reg_def*
Ensure: *reg_seen_defs, paths*

```

1: reg_seen_defs ← new Set()
2: defs_to_check ← new List()
3: defs_to_check.push((reg_def, new List()))
4: seen_defs ← new Set()
5: paths ← new List()
6: while defs_to_check is not empty do
7:   (current_def, current_path) ← defs_to_check.pop()
8:   seen_defs.add(current_def)
9:   current_path.add(current_def.codeloc.block_addr)
10:  def_value ← check_definition_tag(current_def, reg_def)
11:  if def_value is not None then
12:    reg_seen_defs.add(def_value)
13:    paths.add(current_path)
14:  else
15:    for pred in RDA.graph.predecessors(current_def) do
16:      if pred not in seen_defs then
17:        defs_to_check.push((pred, current_path))
18:      end if
19:    end for
20:  end if
21: end while
22: return reg_seen_defs, paths = 0

```

4.5 Backtracking

The data generated by our taint-engine are organized and stored within a DDG, which serves as the foundational structure for all subsequent analyses. Recognizing the need for an effective method to traverse this complex graph, our investigation revealed that a backward or reverse traversal method is often more efficient and requires fewer steps due to the nature of the paths being analyzed. A bottom-up (sink-to-source) approach in graph traversal (which we use in our work) often consumes less memory than a top-down (source-to-sink) method. This efficiency stems from its ability to aggregate results more effectively, thereby reducing the need for extensive state management. Additionally, bottom-up methods may explore fewer nodes, as they can terminate early, once the required information is collected from the sink to the source [14]. Thus, we adopted a customized bottom-up depth-first traversal algorithm, which is particularly advantageous as it allows us to selectively visit only the definitions that are relevant to the specific analysis at hand. For instance, this tailored traversal method is crucial when tracking tainted paths from a sink to its source (e.g., NVRAM_Set to NVRAM_get, system to get_cgi), or when inspecting potential sanitization measures. The specifics of this customized bottom-up depth-first traversal are detailed in Algorithm 1 inspired by [8]. Given a definition, the algorithm uses the DDG associated with the RDA to recursively traverse backward through its predecessor definitions until it either encounters a definition with a ReturnValueTag or exhausts all predecessor definitions. Upon finding a definition with a ReturnValueTag, it returns relevant information associated with both Sink and Source, and the list of related connected definitions.

4.6 Firmware Related Functions Emulation

There are four types of web-based firmware-function calls within the inspected firmwares' binary files that we have on the radar: 1) Functions that receive input from the user over a web interface, 2) functions that store *key: value* input into shared memory, 3) functions that retrieve *key: value* input from shared memory, and 4) functions that perform inspected Sinks (e.g., system, strcpy).

4.6.1 NVRAM Functions Handling. We start by collecting all well-known NVRAM functions that retrieve values using NVRAM Getters (e.g., *nvr_get*) or store values using NVRAM Setters (e.g., *nvr_set*), influenced by the relevant literature contributions (e.g., SaTC and Karonte), vulnerability discovery reports, and our manual inspection of new targeted firmware images. Based on their name and prototype, we implement their related function handlers in order to perform their tainting effects, when encountered, on the DDG generated by the reaching definitions taint-engine during the analysis. We emulate Getters (e.g., *PuVar5* = *nvr_get*("keyword")) similarly to how we handle direct input receiver functions (e.g., *PuVar1* = *get_cgi*("keyword")). However, emulating Setter functions (e.g., *PuVar5* = *nvr_set*("key", *Value*)) involves additional steps to accurately emulate their behavior. Initially, upon encountering a Setter function, we assess its second parameter. If this parameter is a constant, we discontinue processing at this point and proceed with the Reach Definition generation from the next instruction. Conversely, if the second parameter is a string variable, we delve deeper utilizing our backtracking Algorithm 1 described in Section 4.5 to inspect its definition. Based on the algorithm output, we gather the required details that connect both the Source and the Sink such as the keywords, function names, and related instruction addresses and record them into *File_get_2_set*. The gathered data is instrumental during the post-processing phase, as outlined in Section 4.9.

4.6.2 User Input Receiver Functions. We systematically catalog key functions known to accept user inputs, as highlighted in the relevant literature and vulnerability discovery reports (e.g., *websGetVar*(*param1*, "keyword", *acStack256*)). Subsequently, we develop specific handlers to accurately simulate their operational behaviors. Our analysis reveals that functions processing user-controlled inputs fall into three primary categories: The first category involves functions that accept a single keyword and return its associated value to a return register (e.g., *PuVar1* = *get_cgi*("keyword")). To emulate these functions during reach definition generation, we establish a handler for each one. This handler determines the constant parameter's definition, assigns a new definition to the return register, marking it with a ReturnValueTag—a critical marker for halting navigation within the DDG. Furthermore, the corresponding handler creates an edge from the constant definition to the new return register definition.

The second category encompasses functions that receive a keyword as a second parameter and utilize a stack or global buffer as a third parameter, in order to store user inputs based on the specified keyword into the designated memory location (e.g., *websGetVar*(*param1*, "keyword", *acStack256*)). For these functions, we implement handlers that, upon activation during Reach Definition generation, retrieve the constant parameter's definition, update or

establish a new definition for the memory buffer (specified as the third parameter), and attach a ReturnValueTag with an additional field to maintain the keyword's integrity.

The third category includes user-defined functions that often have unique, randomly generated names resulting from the disassembly process, which may vary across different firmware images. These functions typically exhibit the operational patterns of the first two previously mentioned categories. Therefore, we leverage their prototypes. During our reaching definition analysis, these third-category functions are treated as locally defined functions. When such functions are detected, the `handle_local_function` method in the `angr RDA_handler` is triggered. To manage these effectively, we implement an automated process to pinpoint user-defined functions that receive input from end-users, based on specific keywords. We observe multiple common input patterns (e.g., `Var= FUN_110c("keyword"), uVar1 = FUN_00425918(param, "ping_times"), FUN_17190(param, "DNS", local_81c, 0x800)`).

While this automated strategy effectively identifies numerous new taint sources, it is not without its limitations, primarily the generation of false positives. To mitigate this, we implemented a post-processing procedure aimed at eliminating erroneously identified taint sources. This process hinges on the premise that genuine taint sources are typically utilized across multiple functions, whereas false positives have limited usage. Our methodology involves: 1) Computing the frequency of each function, 2) Identifying the top-three most commonly used functions (based on our experiments, selecting the top three limits the occurrence of false positives), and 3) optionally, setting a threshold based on their relative frequency for function reporting. To formalize this procedure, we consider the following:

Let $K = \{k_1, k_2, \dots, k_m\}$ represent the set of functions.

Define $C(k)$ as the count of function k in K , and

$T = \sum_{k \in K} C(k)$ as the total count.

Given a threshold percentage p , select k such that $\frac{C(k)}{T} \times 100 \geq p$.

Output the top n functions based on $C(k)$ meeting p .

In our analysis, we used 10, 15, and 20 as a threshold, and 20 was selected, since higher values did not further improve the results. This procedure, balancing automated identification with critical post-processing, offers a good strategy for taint-source discovery.

4.6.3 Sink Call Handling. We detail next our method for addressing sink calls, which are pivotal in the context of vulnerability discovery such as command injection and buffer overflow. Our initial step involves compiling an extensive list of critical functions highlighted in the literature, vulnerability reports, or identified through our own detailed analysis (e.g., `system`, `do_System`, `wl_exec_cmd`, `strcpy`, `sscanf`, `sprintf`), which are treated as dynamically linked functions within our reaching definition analysis. To manage these functions effectively, we have developed specific handlers within the `RDA_Handler`. The handling process begins by examining the targeted variable associated with a given sink call. If the targeted variable is constant, the process proceeds by generating subsequent Reach Definitions. However, if the variable in question is a string variable, our strategy involves a deeper examination using our

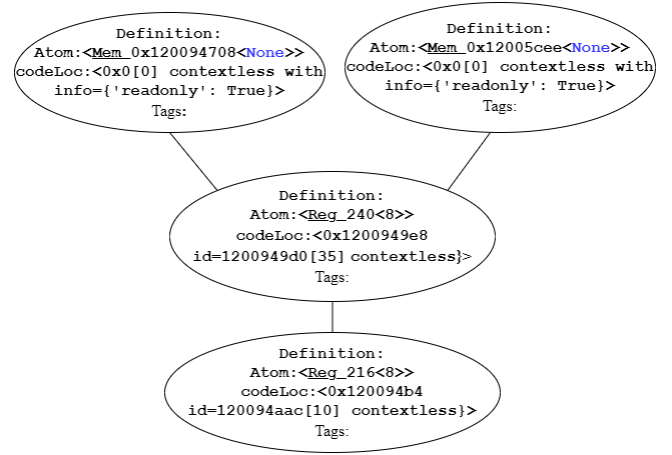


Figure 2: DDG of Indirect Call Depicted in Listing. 3 Line 15

sophisticated backtracking algorithm (described in Algorithm 1, Section 4.5). This detailed analysis enables us to extract vital information that links the source and the sink, such as keywords, function names, and related instruction addresses. This information is then documented in `File_get_2_Sink`, which becomes a crucial asset during the post-processing stage, as detailed in Section 4.9.

4.7 Indirect Call Handling

Our examination encounters two distinct scenarios. The simpler scenario involves easily identifying the target function's address in the call instruction, which is often found in the last instruction of a clinic related block. This is a common occurrence with `libc` functions (e.g., `strcpy`) when they are called indirectly.

Listing 3: Indirect Call Example Decompiled by Ghidra

```

1 void NK_UILanguage() {
2     code = pcVar2;
3     char S0[520], S1[128], S2[520];
4     ..
5     if (extraout_v0 == 1) {
6         NK_query_entry_get("languageName", S0, 0x200);
7         strcpy(S1, S0);
8         pcVar2 = CreateLanguage;
9     }
10    else if (extraout_v0 == 2) {
11        NK_query_entry_get("languageName", S2, 0x200);
12        strcpy(S1, S2);
13        pcVar2 = ModifyLanguage;
14    }
15    (*pcVar2)(S1);
16 }
17 void CreateLanguage(param1) {
18     ..
19     sprintf(&S1, "mv -f /tm/checkFile /etc/%s", param1);
20     system(stack0xffff);
21     ..
22 }

```

Conversely, there are instances where clinic cannot ascertain the exact destination of the function call due to multiple potential targets. Such an example is depicted in Listing 3, line 15. In this example, the target of the call instruction is identified as a register (e.g., `jalr t9` in MIPS). The definition of `t9` generated by RDA is depicted in Figure 2 (Definition Atom: <Reg 216>) where 216 is the `t9` register number assigned by the `angr` framework. In

order to infer the addresses of the potential target functions, in the presence of an indirect call, we employ backtracking over the RDA DDG. We implement a customized version of our previously detailed backtracking Algorithm 1. In the context of Listing 3, the modified algorithm starts the process from the definition of Register 216 (the call register) at line 15 and gathers all instances of read-only memory location definitions where their addresses are function addresses (e.g., `CreateLanguage` and `ModifyLanguage`). Subsequently, the algorithm preserves the current state of RDA for multiple uses. Following this, it iterates over each function address that we have recovered earlier from the DDG and subsequently reconstitutes the associated CFG, and delegates further processing to the `handle_local_function` handler. This handler then proceeds with the RDA process, taking into account the current state and the newly formed CFG. It is important to mention that our indirect call handling does not exhibit indirect calls false positives, but it may miss an indirect call where the function address is derived from runtime evaluations on indirect memory operations, rather than being stored in registers. However, this type of scenario is rare, occurring in approximately 1% of the instances within the *EmTaint* dataset. Moreover, we emphasize that the aforementioned iteration is not time-consuming. Our process only iterates over a few memory addresses residing up to two levels up from the call parameter definition in the generated DDG (as outlined in Figure 2). For example, in Listing 3, *OctopusTaint* iterates only over two functions, `CreateLanguage` and `ModifyLanguage`. The function address lookup is verified directly in the function module dictionary of the *angr* framework. We also note that *SaTC* does not handle indirect calls. However, *EmTaint* proposes a procedure using Structured Symbolic Expression (SSE).

4.8 Sanitization Inspection

In this section, we outline how we check for the existence of security checks that the developers could implement in order to force the correct format of the data inputs received from the end-user. Given a path (resulting from the backtracking Algorithm 1) along with its related details (as outlined in Section 4.5), we first extract the initial basic block, where the tainted source is located, and the concluding basic block, where the tainted sink is found, by using the definitions gleaned from the DDG. We then implement a backtracking procedure that takes the starting and ending basic block addresses and utilizes the Inter-procedural Control Flow Graph (ICFG) and DDG to traverse the ICFG in reverse using the bottom-up depth-first traversal algorithm. This process ultimately yields a list of the tainted path's basic blocks. On this list we scrutinize each basic block to determine if its final statement is a condition. If it is, we extract each condition operand-related definition from the DDG. In addition, we extract any kind of error handling process that may exit (e.g., an error message, an exit statement in case where the extracted condition is not satisfied). Our sanitization inspection methodology systematically examines one of three key avenues to check whether a tainted input undergoes an effective sanitization process. The initial avenue involves scrutinizing whether the input definition undergoes evaluation compared to a constant integer or an alternative definition. We expressly exclude comparisons with `0x0` from being

categorized as sanitization processes. As such comparisons predominantly serve to ascertain the non-nullity of the definition rather than its sanitation. The second check evaluates whether the input definition is subjected to any operations that inherently limit its length. This includes situations where the input is restricted at the source, exemplified by Listing 4 at line 3, with `0x800` representing the maximum permitted length. It also encompasses its processing through secure string manipulation functions (e.g., `strncpy`, `strncat`, `snprintf`, `strncpy`, `memcpy`, `memmove`), provided the specified length is not derived from the scrutinized, tainted input source. The third check assesses the interaction of the input definition with functions recognized as validators. An example of such a function is `isValidIpAddr` or `isLanSubnet`, which returns a flag later examined to validate the input. Failure in this validation typically results in error messages or alternatively tailored exit or debug functions, as outlined in Listing 4 at lines 6-12. Additionally, functions ensuring that the input conforms to an integer format, (e.g., `atoi`), are also considered under this category of sanitization mechanisms. All this information and the associated tainted path are recorded for subsequent analysis.

Listing 4: Sanitized Inputs Example Decompiled by Ghidra

```

1  void FUN_0004aa00 () {
2      ..
3      FUN_000184d4(param1, "wan_ipaddr", S0, 0x800);
4      FUN_000184d4(param1, "wan_netmask", S1, 0x800);
5      FUN_000184d4(param1, "wan_gateway", S2, 0x800);
6      iVar2= inet_addr(S0);
7      iVar3= inet_addr(S1);
8      iVar4= isValidNetmask(S2);
9      if (iVar4 == 0) {
10         iVar5= "invalid_mask";
11         if ((iVar4 ^ iVar2) & iVar3 == 0) {
12             if (iVar2 != iVar4) {
13                 acosNvramConfig_set("wan_ipaddr", S0);
14                 acosNvramConfig_set("wan_netmask", S1);
15                 acosNvramConfig_set("wan_gateway", S2);
16             }
17         }
18     }
19     ..
20 }
21 void FUN_00010acc () {
22     ..
23     uVar2=acosNvramConfig_get("wan_ipaddr");
24     uVar3=acosNvramConfig_get("wan_ifname");
25     sprintf(S0, "route add -host %s dev %s", uVar2, uVar3);
26     system(S0);
27     uVar4=acosNvramConfig_get("wan_gateway");
28     sprintf(S2, "route add -host default gw %s", uVar4);
29     system(S2);
30 }

```

Finally, it is important to highlight that sanitization should ideally be implemented at the point of initial data reception from the end-user, as demonstrated in Listing 4 at lines 6-13. Once this initial sanitization check is conducted, subsequent rechecks are unnecessary. Therefore, when the previously verified information is reused, there is no need for additional inspection, as indicated in Listing 4 at lines 23-29. This strategy ensures that once data is deemed safe, it can be efficiently utilized without the redundancy of further checks. Our post-processing filtering procedure outlined in Section 5.4 takes this information into consideration when it connects the execution paths to avoid false positives.

4.9 Candidate Path Generation

At this stage, we operate through a series of rounds, iterating over each reported path contained within every `File_get_2_Sink` file. Our primary objective is to ascertain whether the source of the current path is either a recognized shared source (e.g., `nvrw_get`) or an interface designed to receive user input (e.g., `get_cgi`). When the source is identified as originating from a user input getter (e.g., `get_cgi`), and its corresponding token aligns with keywords extracted from one of the web interfaces, we classify the path as a potential vulnerability of the direct type. Conversely, if the source traces back to a location such as `nvrw_get`, *OctopusTaint* initiates a search within stored paths. This search encompasses both the `File_get_2_Set` resulted from the same binary and potentially extends to `File_get_2_Set` resulting from other binaries that share memory with an identical token. This exploration proceeds recursively, especially in the cases where transitive NVRAM set operations come into play. The recursion persists until we either reach the initial source responsible for receiving user input or conclude that the provided inputs do not originate from a user-provided source. In scenarios where it is plausible to establish a connection between paths based on shared memory tokens, we generate an alert classified as indirect vulnerability type. This alert includes all pertinent details related to the potential issue. Conversely, if no viable connection between paths exists, we opt to disregard these paths.

5 Evaluation

In this section, we comprehensively evaluate *OctopusTaint* using three distinct datasets, which are elaborated upon in Section 5.1. Our objective is to address the following research questions:

- **Q1:** How effective is *OctopusTaint* in identifying vulnerabilities?
- **Q2:** What is the efficacy of *OctopusTaint* in tracking tainted inputs across NVRAM related system calls?
- **Q3:** To what extent does our novel filtering strategy decrease the incidence of false positives?
- **Q4:** How effectively does our system identify new sources of input taint?
- **Q5:** What are the benefits of integrating the clinic Module with RDA?
- **Q6:** How does *OctopusTaint* compare in terms of performance to existing state-of-the-art approaches?

5.1 Datasets

We collected three datasets for our analysis to address the research questions outlined earlier.

- (1) Our dataset: We collect the an up-to-date set of firmware to demonstrate the efficacy of *OctopusTaint* in identifying zero-day vulnerabilities within the latest firmware versions. This set is composed of 160 firmware images from major router vendors, including *ASUS*, *Linksys*, *Netgear*, and *TP-Link*.
- (2) *SaTC* dataset [6]: It is comprised of 39 firmware images and was selected to facilitate direct comparison between the *OctopusTaint* and *SaTC* methodologies. Besides, the *SaTC* dataset includes 39 verified vulnerabilities.
- (3) *EmTaint* dataset [7]: It is composed of 35 firmware samples from six vendors, including *D-Link*, *TP-Link*, *Netgear*,

TRENDnet, *Cisco*, and *Tenda*. In addition to the firmware samples, this dataset is accompanied with a list of tainted sink addresses reported in the results generated by *EmTaint*. Besides, it includes 192 verified vulnerabilities. Also, according to *EmTaint* [7], many of the reported tainted sinks have indirect calls and alias variables.

- (4) Large Scale Dataset [15,16]: We utilize a recent public firmware dataset², which has a total size of 6 TB. This dataset includes firmware from various operating systems, such as *Linux*, *eCos*, *RTOS*, *VxWorks*, and *pSOS*. It has samples from 100 vendors. We specifically selected this dataset to evaluate *OctopusTaint* on a richer dataset. To ensure a representative distribution, we randomly selected up to 100 firmware samples from each vendor, depending on availability, ensuring each vendor's presence. Our analysis targeted firmware that provides a web interface, with the selected firmware commonly used in VPNs, cameras, routers, TV-IP, WiFi extenders, and smart home devices.

5.2 Identification of Vulnerabilities

This section provides a detailed capability evaluation of *OctopusTaint* to identify zero-day vulnerabilities within our rich and diverse dataset. Table 1 illustrates the systematic process of *OctopusTaint* as it filters through the data to pinpoint vulnerabilities with considerable exploitation potential. We specifically focus on those devices for which *OctopusTaint* generates high-confidence alerts, ensuring clarity and relevance of the data presented. Table 1 illustrates the effective filtration mechanism, which efficiently reduces the initial volume of thousands of alerts to a refined set containing the most exploitable ones, as explained in Section 4.8 and Section 5.4. For example, while analyzing the firmware *Netgear-r6400-1.0.1.78*, *OctopusTaint* initially identified 33,292 alerts but refined these to 816 unique candidates following a robust sanitization process. Subsequent post-filtering procedures further narrowed this down to 109 critical alerts. These alerts, stemming from a dedicated user interface form and lacking effective sanitization, include 47 Type-1, 64 Type-2, and notably, 47 that are classified as transitive Type-2. The final refined set includes 5 command injections and 106 buffer overflow alerts, demonstrating the precision of *OctopusTaint* in isolating the most significant vulnerabilities. In contrast to earlier methods such as *EmTaint*, which often generate numerous alerts necessitating extensive manual review and a high likelihood of false positives, *OctopusTaint* employs an automated system that enhances efficiency and accuracy. It rigorously checks for the absence of sanitization and applies a series of sophisticated filtering procedures. This ensures that the alerts flagged are free from sanitization and originate from user-controlled inputs, significantly reducing the need for manual intervention and minimizing the risk of false positives.

5.2.1 Accuracy Evaluation. To evaluate the accuracy of *OctopusTaint*, we conducted a manual inspection of 100 randomly selected samples from the candidates listed in Table 1. This evaluation methodology aligns with those employed in previous studies on *SaTC* and *EmTaint*. As indicated in the True Positive (TP) column

²<https://github.com/WUSTL-CSPL/Firmware-Dataset>

Firmware	Alerts	Sanitized	Sinks	Candidates	Direct	Indirect	Transitive	CI	BoF	TP	FP
Linksys-e1200-2.0.11	78769	22712	219	11	2	9	0	0	11	6	3
Linksys-e900-1.0.09	141739	35214	237	3	1	2	0	0	3	2	1
Netgear-ex6120-1.0.0.68	14258	756	136	10	1	10	0	3	7	5	4
Netgear-nbk752-4.6.5.11	38986	6797	501	24	4	20	0	1	23	-	-
Netgear-r7000-1.0.11.134	10863	2368	494	40	27	13	1	2	38	11	10
Netgear-r6400-1.0.1.78	33292	7213	816	109	45	64	47	3	106	9	19
Netgear-ex6100-1.0.0.68	23554	728	299	54	19	35	0	5	49	16	14

Table 1: Results of *OctopusTaint* Execution over our Collected Dataset

of Table 1, 49 of the manually inspected candidates were confirmed as true positives. Each true positive sample meets the following criteria: (1) the tainted input originates from a user-controllable web interface, (2) the tainted input is received at the back-end by a recognized taint source receiver based on the specified keyword, and (3) the tainted input propagates to a dangerous sink directly or indirectly without encountering any sanitization measures. The types of vulnerabilities identified by *OctopusTaint* have serious security implications for the targeted IoT firmware. For example, OS Command Injection vulnerabilities (CWE-78) enable arbitrary command execution. Buffer overflow vulnerabilities (CWE-119 and CWE-121) result in arbitrary code execution. Format string vulnerabilities (CWE-134) cause information disclosure. Regarding the prevalence of vulnerabilities, of the 49 confirmed cases, three are command injections, while the rest are buffer overflows. Among these, seven are direct (type-1), three are indirect and transitive (type-2), and the remaining are indirect (type-2) based on our classification in Section 2.2. Notably, the indirect and transitive (type-2) vulnerabilities are only observed in *Netgear* router firmware. The other types are prevalent across various devices from different vendors.

5.2.2 False Positive Analysis. Our detailed verification process identifies certain scenarios where the alerts generated by *OctopusTaint* are false positives. Specifically, some execution paths leading to the tainted sources are determined to be infeasible. An example of such a scenario is presented in Listing 5. In this example, the input is received from the end user at line 3 and at line 5 passed to the `legal_hwaddr` function, which verifies the validity of a given MAC address format. *OctopusTaint* misses this sanitization step because its inspection mechanism only evaluates conditions within basic blocks between the source and the sink. However, when tainted input is validated in a separate function without altering its definition, *OctopusTaint* misses it.

Listing 5: Unknown Sanitizer Function

```

1 void hnd_parentalctrl_unblock(){
2     ..
3     pu = get_cgi("HND_block_mac");
4     ..
5     iVar6 = legal_hwaddr(pu);
6     if (iVar6 != 0){
7         ..
8         nvram_set("hnd_unblock_mac", pu);
9         snprintf(ac, 0x200, "echo \"%s\" > /proc/unblock_proc", pu);
10        system(ac)
11    }
12 }
13 bool legal_hwaddr(char *param1){
14     if (Mac address is valid) return true;
15     else return false;
16 }

```

The second scenario where *OctopusTaint* reports a false positive occurs when tainted inputs are inspected earlier in the execution path than the reported candidate path. An example of such situation is presented in Listing 6. In this example, the input is received from the user at line 4 and then saved into NVRAM in line 5. At line 15, within the `wan_down` function in another binary file, the tainted input is recovered from NVRAM, then propagated into `sprintf`, and finally into a system call. The false positive arises because *OctopusTaint* is unaware that the tainted input has already been inspected and validated earlier at lines 11 and 12.

The third scenario encountered involves *OctopusTaint* not recognizing that the tainted input has been passed through a function that converts string to long, float, or enforcing numerical format through format strings as with `sprintf`. The current implementation does not consider all numerical conversion function variations. In practice this could cause the software to crash. However, we currently consider these incidences as a false positive and we will implement the additional logic that fixes the aforementioned issue, in the next version of *OctopusTaint* tool.

Listing 6: Earlier Tainted Input Inspection

```

1 // in httpd binary file
2 void basicCgiPtpMain(){
3     ..
4     webGetVar(param1, "pptp_serv_id", local_1c0);
5     acosNvramConfig_Set("pptp_serv_id", local_1c0);
6     ..
7 }
8
9 // in ledgreen binary file
10 bool wan_down(){
11     uvar1 = acosNvramConfig_get("pptp_serv_id");
12     uvar3 = isValidIpAddr(uvar1);
13     if (ivar3 != 0){
14         ..
15         uvar1 = acosNvramConfig_get("pptp_serv_id");
16         uvar9 = acosNvramConfig_get("pptp_gateway");
17         sprintf(ac, "route add -host %s gw %s", uvar1, uvar9);
18         system(ac)
19     }
20 }

```

5.3 NVRAM System Calls Tracking

We evaluate over our dataset the Type-2 Vulnerabilities, as elaborated in Section 2.2. Although this type of vulnerability is partially addressed in *SaTC* and *EmTaint*, these solutions lack comprehensive coverage of transitive vulnerabilities—a critical aspect that our in-depth analysis highlights, particularly in *Netgear* router devices. Notably, our investigation uncovered a large number of transitive vulnerability instances: 47 cases in *Netgear* R6400 and 1 in *Netgear* r7000, as outlined in Table 1. This emphasis on transitive vulnerabilities sheds new light on the potential security risks in

Firmware	Alerts	With/Without Taint Source Identification				
		CI	BoF	CI	BoF	Candidate
D-Link (6)	9052	5	20	5	35	40
Netgear (13)	154776	1	36	29	509	538
Cisco (2)	15911	0	45	0	45	45
TRENDnet (1)	518	0	1	0	1	1
TP-Link (2)	1667	0	4	0	51	51
Tenda (4)	182931	10	24	14	40	54
Total	376870	16	130	48	661	729

Table 2: OctopusTaint Filtering and New Taint Sources Identification Capabilities

commonly used networking devices, and our research, according to our knowledge, is the first study, that explores this area.

5.4 Post-Processing Filtering Effectiveness

In this evaluation, we use the *EmTaint* dataset and report the candidate paths identified by *OctopusTaint* and not identified by *EmTaint*. An initial observation from column Alerts in Table 2 reveals that *OctopusTaint* identified a substantial total of 376,870 tainted paths. This significant number suggests that in the absence of the sanitation identification and post-filtering strategy, our tool also has a tendency towards over-tainting. That is why *OctopusTaint* includes sanitation and post-filtering by design. It rigorously evaluates whether each path represents a genuine vulnerability. The first stage of this filtering process involves verifying that the taint source based on the extracted keyword originates from a user input form through the web interface rather than from a secure or non-user-controlled location. The second stage assesses the absence of sanitization measures along the inspected path. Only paths that successfully pass these two filters are deemed potentially vulnerable, as explained in Section 4.8. These paths are then recorded for further manual analysis and for preparing Proof of Concept (PoC) demonstrations to establish their validity. As a result of this rigorous post-processing methodology, *OctopusTaint* only recommends 729 candidate paths for manual inspection instead of, as shown in the Candidate column of Table 2.

5.5 New Taint Sources Identification

We conducted two experiments on the *EmTaint* dataset to evaluate the impact of enabling versus disabling the new taint source identification capability in *OctopusTaint* described in Section 4.6.2. The results, as presented in Table 2, clearly demonstrate the significant effect of this capability. With the new taint source identification enabled, the number of candidate alerts reported increased three-fold. Specifically, regarding command injection (CI) vulnerabilities, *OctopusTaint* identified 16 instances when the feature was disabled, compared to 48 instances when it was enabled. Similarly, for buffer overflow (BoF) vulnerabilities, the tool reported 130 instances without the feature and a markedly higher count of 661 instances with it enabled. This stark increase underscores the effectiveness of the new taint source identification in enhancing our tool's ability to detect critical vulnerabilities. Our analysis of the path candidates generated from the *EmTaint* dataset revealed 31 unique new taint sources. Key insights from our findings include the variation in

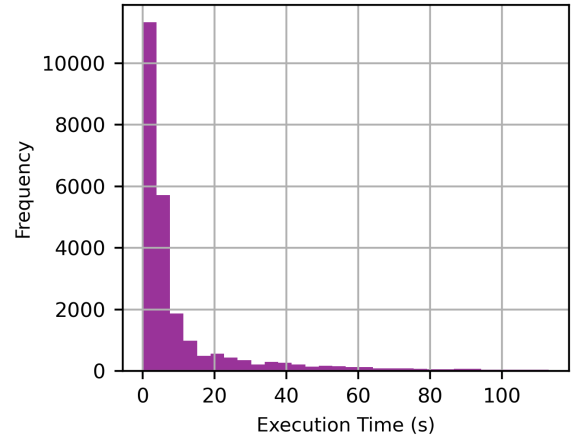


Figure 3: OctopusTaint Execution Times over SaTC dataset

user-input function names across different vendors. For example, in *Netgear*, we identified `sub_18614` in *R6400* and `sub_110ec` in the *R9000*, while in Cisco's routers, we identified `NK_query_entry_get` in the *rv320* and `sub_1d170` in the *rv130*. The disassembly process often introduces random function naming, and within the same firmware, the removal of debugging information can result in the same user-input function having different names across binaries. Consequently, *OctopusTaint* identified three patterns to detect these functions, as outlined in Section 4.6.2.

5.6 Clinic Module Effectiveness

The primary aim of this experiment was to quantitatively assess the impact of integrating the RDA methodology with the clinic module in terms of enhanced identification of tainted candidates. For this evaluation, we used *EmTaint* dataset since it is accompanied by the list of verified tainted sinks' address locations. Therefore, we consider it like a labeled dataset, as detailed earlier. Our findings indicate a significant difference in the outcomes obtained with and without using the clinic module. In the absence of clinic, *OctopusTaint* identified only 275 (20%) candidates, whereas the integration with clinic led to the identification of 1096 (82%) candidates out of the 1338 tainted sinks reported by *EmTaint*, as outlined in Table 6. This significant increase underscores the effectiveness of the clinic module in conjunction with the RDA approach in candidate identification processes. The integration of RDA with *clinic* enhances its capability to trace tainted inputs to precise memory locations. *clinic* assists in returning the memory variable being inspected rather than the value of the register, leading to more accurate localization of memory issues.

5.7 Execution Time Analysis

In our comprehensive analysis of the *OctopusTaint*'s execution times, consisting of 25,442 analyzed functions, we documented a wide range of execution durations, with a mean of approximately 35 seconds and a substantial standard deviation of about 113 seconds. This substantial variability indicates diverse operational behaviors.

Firmware	Alerts	Candidates	Type-1	Type-2		CI	BoF
				Regular	Transitive		
Asus(45)	1678	117	103	14	0	0	117
Linksys(6)	1197	39	26	13	0	6	33
Netgear(49)	24525	2678	879	1799	26	149	2529
Others(46)	20833	1950	715	1235	5	140	1810
Tenda(9)	1312	293	109	184	0	5	288
TRENDnet(8)	120	19	7	12	0	0	19
Conceptronic(4)	10	8	8	0	0	0	8
Swisscom(1)	1	1	1	1	0	0	1
TP-Link(13)	901	195	193	2	0	0	193
Tomato(8)	2448	39	8	31	0	7	32
D-Link(9)	892	25	25	0	0	0	25
buffalo(2)	50	6	6	0	0	0	6
Total	53967	5370	2080	3291	31	307	5061

Table 3: *OctopusTaint* over Dataset 4

To address this skew and enhance the robustness of our analysis, we implemented a Percentile-Based Cutoff at the 95th percentile, established at 113 seconds, as outlined in Figure 3. This methodological choice strategically omits the top 5% of the execution durations, thereby focusing our analysis on the tool's performance under typical operational conditions. The adoption of this threshold is intended to diminish the impact of extreme outliers, which predominantly occur in scenarios involving recursive calls or when RDA component encounters operational halts, necessitating a forced termination after 1000 seconds. This tailored approach ensures a practically adequate and representative depiction of the tool's standard operational capabilities.

5.8 Large Scale Evaluation

In this section, we evaluate *OctopusTaint* using Dataset 4 outlined in Section 5.1. We analyze 1,542 Linux-based firmware that provide a web interface and share keywords between the front-end and back-end. Table 3 summarizes the analysis results for 166 firmware where *OctopusTaint* successfully identifies vulnerability candidates (CI=307 and BOW=5061). Table 3 shows that the number of candidates (5370) is significantly smaller than the number of generated alerts (53967), demonstrating the effectiveness of *OctopusTaint*'s sanitization and post-processing procedures. From the results, we observe several important trends. A notable trend in Table 3 is that almost all reported vendors exhibit direct (does not transit through NVRAM) and regular indirect (transits through NVRAM without keyword change) types of vulnerabilities. However, only firmware images from *Netgear* exhibit indirect and transitive types (transits through NVRAM with keyword change) of vulnerabilities (type-2) within our examined datasets.

Additionally, this study reveals that Linux-based firmware with web interfaces exhibits similar susceptibility to taint-based vulnerabilities, regardless of their category. For instance, the candidate paths recovered from *Netgear* router firmware resemble those recovered from IP-TV cameras from *TRENDnet* or VPN from *Tomato* in terms of taint source, taint propagation, and sink locations. These findings indicate that despite the diversity of devices, the underlying firmware vulnerabilities share common characteristics. Moreover, in our analysis, we find out that *TP-Link* and *D-Link* samples have few or no indirect reported vulnerable candidates. As such, we investigate the samples and find that, over *TP-Link* and *D-Link* sample images, *angr* fails to recover keywords related to Getters

Firmware	OS version	Alerts	Candidates	Direct	Indirect	CI	BoF
TP-Link	5	32	2	2	0	0	2
TP-Link	6	40	2	2	0	0	2

Table 4: *OctopusTaint* over VxWorks-based Firmware

(e.g., `getenv("keyword")`). In fact, as noted by the authors of *EmTaint*, matching the front-end with the back-end using keyword matching as a way to identify potential candidates may lead to missing many potential vulnerabilities. To deal with this issue, *EmTaint* ignores matching the front-end to the back-end using keywords, in contrast to SaTC. With respect to sample images from *TP-Link* and *D-Link*, we follow *EmTaint* and report only direct types of candidates (type-1).

5.9 Non-Linux Firmware Evaluation

To assess the performance of *OctopusTaint* over non-Linux firmware, we decided to opt for *VxWorks*, which is an extremely popular real-time operating system (RTOS) with 23% market share among commercial RTOS. It is deployed on more than 2 billion devices in defence, aerospace, and automotive industries [17]. To properly analyze firmware based on *VxWorks*, we need first to identify the CPU architecture details, locate the load address, and recover the symbol table. To this end, we use *binwalk* to identify the CPU architecture category, bitness, and endianness. Afterwards, we need to identify the load address and recover the symbol table, which is achieved using *vxhunter*³ tool [18]. The latter is built on top of *Ghidra*, and supports only versions 5 and 6 of *VxWorks* OS, which embeds a partial symbol table in the binary [19]. The *vxhunter* tool uses the recovered symbol table to rename all identified functions. Without this renaming process, all identified functions would have random names, including standard library functions. In this pursuit, we collect the necessary details: CPU architecture details, load address, and symbol table information. We use this information to configure the *angr* framework for analyzing the targeted firmware. With this setup, we execute *OctopusTaint* over the targeted *VxWorks*-based firmware functions in a manner similar to our analysis of Linux-based firmware. For example, in the case of a *TP-Link* router based on *VxWorks*, after recovering the necessary information, we find that it uses `websGetVar` to receive inputs over the web interface. However, it employs different *NVRAM* function names, such as `sysNvRamGet` and `sysNvRamSet`, instead of `nvrAm_get` and `nvrAm_set`. The rest of the analysis process remains consistent. As such, we implement their related handlers. Table 4 presents the outcome of *OctopusTaint* analysis on two *VxWorks*-based *TP-Link* routers. The results indicate 72 alerts out of which four potential vulnerable candidates where inputs from user-controllable sources are directly propagated into unsecured `strcpy` functions. We could not evaluate *OctopusTaint* on a large dataset due to the limitation of *vxhunter* tool. It fails to recover the load address, when we provide it with *VxWorks* samples from Dataset 4.

5.10 Comparison to Related Work

We benchmark *OctopusTaint* against *SaTC* [6] and *EmTaint* [7], using their publicly accessible datasets described earlier in Section

³<https://github.com/PAGalaxyLab/vxhunter>

Vendor	Device	SaTC			OctopusTaint		
		BoF	CI	Time	BoF	CI	Time
Netgear	R6400	0	4	30:23h	58	4	17:14h
Netgear	R7000	2	2	11:34h	70	7	21:53h
Netgear	XR300	0	4	22:57h	59	7	21:23h
D-Link	DIR878	0	16	8:22h	6	6	2:26h
Tenda	AC15	0	4	9:23h	7	6	7:50h
Tenda	AC18	0	4	8:25h	7	6	8:36h
Total	-	2	34	92:35h	207	36	70:46hh

Table 5: Comparison of *OctopusTaint* and *SaTC*

Firmware	EmTaint	OctopusTaint		MangoDFA
		Identified	Sanitized	
D-Link (6)	300	249	86	49
Netgear (13)	419	341	124	75
Cisco (2)	457	370	51	7
TRENDnet (1)	11	4	1	0
TP-Link (2)	37	33	19	26
Tenda (4)	114	99	39	18
Total	1338	1096	320	175

Table 6: Comparison of *EmTaint*, *OctopusTaint* and *MangoDFA* on *EmTaint* Dataset

5.1. We did not compare *OctopusTaint* against *Karonte* [5] since *SaTC* demonstrates its superiority over *Karonte* in [6].

Performance Comparison with *SaTC* [6]: To compare with *SaTC*, we selected similar firmware samples *SaTC* used to compare with *Karonte* to establish a fair comparison, the details of which are presented in Table 5. This comparison focuses on two primary evaluation metrics: the number of identified security incidents and the total execution time required for analysis. The data presented in the table clearly demonstrates that *OctopusTaint* matches *SaTC*'s capability to identify command injection vulnerabilities. However, we can see that *OctopusTaint* misses few CI vulnerabilities identified by *SaTC* over *D-Link* samples. To understand why, we conduct a manual analysis. We find out that a limitation in *angr* framework leads to an occasional inability to recover keywords related to Getters (e.g., `getenv("keyword")`), as in the case of *D-Link* binary files. As a result, *OctopusTaint* ignores related path candidates since it cannot connect the front-end and the back-end. In this comparison, the reported results are automatically generated by *OctopusTaint* with high confidence since the inspected tainted path originates from a user-controllable input.

On the other hand, *OctopusTaint* excels notably in detecting buffer overflow incidents, outperforming *SaTC* by a substantial margin. This success is largely attributed to the RDA integration with the clinic module, which provides *OctopusTaint* with the enhanced ability to precisely recover and scrutinize tainted stack variables, as well as to adeptly address indirect call challenges. As a result, *OctopusTaint* is able to explore areas of the code that *SaTC* does not cover. In terms of execution time, *OctopusTaint* demonstrates superior efficiency, requiring only 70 hours and 46 minutes to analyze the selected firmware samples, whereas *SaTC* requires

92 hours and 35 minutes to accomplish the same task. This marked improvement in analysis speed, alongside the comprehensive coverage, underscores the enhanced efficiency and performance of *OctopusTaint* when juxtaposed with current *state-of-the-art* solution. Therefore, the empirical evidence from our evaluation suggests that *OctopusTaint* not only achieves more thorough vulnerability detection but also represents a more time-efficient approach to firmware analysis. Moreover, It is worth mentioning that for fairness, we have taken into account the time delay introduced by the freezing or cycling situations outlined in Section 5.7. We also note that the large number of identified BoF by *OctopusTaint* could be lower since many of them shared the taint source. According to CVE records, Sinks that share the same taint are counted as one. From the CVE records related to firmware, sometimes seven sinks share one source. We could not run *EmTaint* approach over *SaTC*. *EmTaint* uses IDA Pro [20] to disassemble and build the CFG of the targeted binaries. We contacted their team hoping for a solution but received no answer. On the other hand, we could not run *SaTC* over the *EmTaint* dataset because *EmTaint* published only `httpd` and `CGI` binary files without providing the related web interface, which is necessary for the *SaTC* approach.

Performance Comparison with *EmTaint* [7]: We run *OctopusTaint* over *EmTaint* dataset and report the results in Table 6, which reveals the following: Out of the 1,338 unique tainted sinks reported by *EmTaint*, *OctopusTaint* identifies 1,096 candidate paths. *OctopusTaint* could not reach 242 tainted sinks because reaching them involves handling indirect calls where function addresses are derived from runtime calculations on indirect memory operations, rather than being stored directly in registers. On the other hand, *OctopusTaint* identifies sanitization presence over 320 sinks reported by *EmTaint*. To verify the sanitization inspection capability within *OctopusTaint*, we conduct a manual verification over sanitized instances reported by *OctopusTaint*. Listing 4 provides an example of a candidate reported by *EmTaint* and identified as a sanitized instance by *OctopusTaint*. We investigate half of the 320 instances, and we confirm the presence of sanitization code in 104 instances, as outlined in Section 4.8 and Section 5.4. For the remaining instances, we find that the tainted input is guarded with complex conditions making the exploitation of the corresponding paths unlikely. Besides, it is worth mentioning that, *EmTaint* only checks for the length of the given tainted input. It does not consider inspecting the presence of security checks, filtering, and/or comparison functions (e.g., `strcmp`, `atoi`, `isValidIpAddr`) as *OctopusTaint* does. In addition, further scrutiny of the data presented in the last column of Table 2 reveals additional strengths of *OctopusTaint*. Notably, *OctopusTaint* identifies an additional 729 candidate paths that were not detected by *EmTaint*. The discovery of these additional candidate paths is made possible due to *OctopusTaint* capability to identify new taint sources, which enables *OctopusTaint* to explore areas overlooked by *EmTaint*. We manually inspect 50 randomly selected candidate samples from *D-Link* and *Netgear* firmware images. We then verified 23 true positives out of the 50 candidates. On the other hand, we encountered false positives that fall within the scenarios discussed in Section 5.2.2.

Performance Comparison with *MangoDFA* [10]: We run *MangoDFA* on the labeled *EmTaint*'s dataset and report its performance in the last column of Table 6. As shown in Table 6,

Approach	Methodology	Taint Tracking		Direct (type-1)		Indirect (type-2)		Sanitization	Indirect Call	User Input Receivers	Parameters Recovery	Vulnerabilities				OS					
		Source2Sink	Sink2Source			Regular	Transitive									CWE-78	CWE-121	CWE-119	CWE-134	Linux	VxWorks
Karonte [5]	DSE	•		•		•							•			•					
SaTC [6]	DSE	•		•		•							•			•					
EmTaint [7]	SSE	•		•		•			•		•	•	•	•	•	•					
MangoDFA [10]	RDA		•	•		•						•	•			•					
OctopusTaint	RDA		•	•		•	•	•	•	•	•	•	•	•	•	•	•				

Table 7: A comparison of state-of-the-art related approaches. (•) means that the approach provides the corresponding feature

MangoDFA identifies 16% of the candidates labeled by *EmTaint*, whereas *OctopusTaint* identifies 83%. The lower accuracy of *MangoDFA* can be attributed to several factors: 1) the dataset contains an abundance of indirect call sites (e.g., Cisco routers), 2) the dataset includes several new taint sources unknown to *MangoDFA* (e.g., `NK_query_entry_get` in the *rv320* router), 3) the dataset encompasses new types of taint-based vulnerabilities uncovered by *MangoDFA* (e.g., CWE-134), and 4) *MangoDFA*, built on top of the *anqr* framework, sometimes fails to recover function parameters when more parameters are involved than available registers (e.g., `sprintf` and `sscanf`). To overcome these limitations, *OctopusTaint* is designed to address these challenges. The objective of *OctopusTaint* is to create a static-based approach that matches the effectiveness of *state-of-the-art* symbolic-based approaches (e.g., *EmTaint* [7]).

6 Discussion

We compare *OctopusTaint* against *state-of-the-art* approaches, considering various factors, including methodology, taint tracking direction, types of vulnerabilities, sanitization inspection, indirect call handling, recovery of function parameters, vulnerability categories, and the targeted firmware operating systems. Table 7 summarizes these aspects of current *state-of-the-art* approaches. The comparison reveals that *OctopusTaint* uniquely addresses sanitization inspection, identification of user-defined input receivers, identification of indirect and transitive vulnerabilities, and exploration of non-Linux firmware. From Table 7, we observe that *Karonte*, *SaTC*, and *EmTaint* employ methodologies based on DSE and tracking of tainted inputs from source to sink.

Conversely, *MangoDFA* and *OctopusTaint* base their methodologies on reaching definition analysis and tracking of tainted inputs from sink to source. While *MangoDFA* and *OctopusTaint* share some features, it is important to note that *OctopusTaint* was independently developed from *MangoDFA*, which was made public just two weeks before *OctopusTaint*. *MangoDFA* is a significant contribution that demonstrates the scalability of reaching definitions. However, *OctopusTaint*, beyond reaching definition analysis, also

introduces unique and novel contributions, including sanitization inspection and post-filtering procedures, which are synergistically combined to significantly reduce false positives and enhance accuracy. Additionally, *OctopusTaint* tracks tainted input over NVRAM, identifying type-2 vulnerabilities—i.e., indirect, transitive vulnerabilities—as detailed in Section 2.2. It also identifies user-defined taint source functions, facilitating the discovery of new taint sources. Furthermore, *OctopusTaint* implements function parameter recovery, helping identify format string vulnerabilities in functions such as `sprintf` and `sscanf`, which may be called with more parameters than the available registers. From Table 7, we observe that only *EmTaint* and *OctopusTaint* handle indirect calls. *EmTaint* addresses it using Structured Symbolic Expression (SSE), whereas *OctopusTaint* employs reaching definitions and backtracking, as outlined in Section 4.7. Regarding identified vulnerability types, from Table 7, we can see that *Karonte*, *SaTC*, and *MangoDFA* only cover command injections (CWE-78) and buffer overflows (CWE-121). Only *EmTaint* and *OctopusTaint* additionally cover format string vulnerabilities (CWE-134) and buffer overflow vulnerabilities (CWE-119).

7 Related Work

We examine existing literature addressing taint-based vulnerability detection in Linux-based IoT devices that feature web interfaces. The primary methodologies can be broadly classified into static-based and dynamic-based analysis. Static-based approaches, in comparison to dynamic ones, offer greater scalability but often suffer from a higher rate of false positives. Conversely, dynamic-based methods, such as fuzzing, typically yield a higher true positive rate but are computationally demanding, making them less practical for large-scale application. Furthermore, dynamic solutions [21, 22] involving IoT firmware simulation and re-hosting face applicability issues across diverse devices, especially since many vendors develop their customized versions, as discussed in [23].

One approach, *Karonte*, tracks user-controlled inputs transmitted via HTTP headers, such as environment variables, across multiple backend binaries by constructing a multi-binary graph [5]. It uses

DSE to determine whether tainted inputs can reach critical sinks. In a related vein, *SaTC* [6] adopts a different strategy by analyzing unpacked firmware to extract web interface files, including HTML, XML, and Ajax documents. It identifies keywords corresponding to backend API handlers and employs unconstrained symbolic execution to explore whether there exists a feasible path that can propagate user inputs to critical system functions (e.g., `memcpy`, `strlen`, `system`). The tool ultimately generates a detailed report of the entire tainted path from the input to the sink, facilitating manual verification by security analysts. Despite the promising results reported by *Karonte* and *SaTC*, both approaches exhibit significant limitations: They utilize symbolic execution to analyze all potential paths, which can be time-consuming, especially for larger firmware sizes. They tend to generate a high number of false positives, complicating the validation process and potentially leading to overlooked vulnerabilities. In the same direction but using a different approach, *EmTaint* proposes SSE-based alias analysis to resolve indirect calls while searching for taint-based vulnerabilities. As a result, *EmTaint* is able to explore deeper paths that were overlooked by previous approaches such as *SaTC* and *Karonte*.

HermeScan [24] and *MangoDFA* [10] are based on Def-use and built a top DDG. *HermeScan* adopts a taint tracking methodology akin to DSE, pursuing tainted inputs in a forward direction. To mitigate path explosion, *HermeScan* incorporates a path-merging strategy reminiscent of enhancements traditionally applied to DSE. Conversely, *MangoDFA* employs a reverse traversal algorithm, moving from sink to source, to circumvent the path explosion problem and enhance the speed of analysis, particularly on a larger scale. Despite their advancements, both tools fall short in their treatment of alias analysis and their scrutiny of sanitization practices.

8 Conclusion

In this paper, we present *OctopusTaint*, a robust and fully static taint-based data flow analysis approach, specifically designed and implemented to improve scalability and minimize false positives, featuring advanced mechanisms to scrutinize sanitization measures within tainted paths—common sources of false positives in similar tools. Our evaluation highlights *OctopusTaint*'s capability in handling indirect calls, aliasing, and sanitization inspections, which are essential for a thorough analysis of vulnerabilities. Notably, *OctopusTaint* not only matches but also surpasses current *state-of-the-art* solutions, *SaTC* and *MangoDFA*, demonstrating superior efficiency and accuracy. In our tests, *OctopusTaint* successfully identified 142 previously reported *n*-day vulnerabilities by *SaTC* and *EmTaint*. Moreover, *OctopusTaint* discovered several unreported 0-day vulnerabilities. Comparative evaluations also reveal that *OctopusTaint* outperforms *EmTaint*—another leading tool that utilizes SSE for addressing indirect calls and aliasing. Specifically, *OctopusTaint* achieved an accuracy rate of 82% on *EmTaint*'s dataset and proved more effective in inspecting sanitization, identifying 320 sinks as sanitized, which *EmTaint* had flagged as genuine alerts. Additionally, *OctopusTaint* detected extra vulnerabilities missed by *EmTaint* demonstrating its advanced capabilities in detecting new taint sources.

References

- [1] X. Yin, R. Cai, Y. Zhang, L. Li, Q. Yang, and S. Liu, "Accelerating command injection vulnerability discovery in embedded firmware with static backtracking analysis," in *Proceedings of the 12th International Conference on the Internet of Things*, 2022, pp. 65–72.
- [2] K. Cheng, D. Fang, C. Qin, H. Wang, Y. Zheng, N. Yu, and L. Sun, "Automatic inference of taint sources to discover vulnerabilities in soho router firmware," in *IFIP International Conference on ICT Systems Security and Privacy Protection*. Springer, 2021, pp. 83–99.
- [3] SAM-IOT-Security-Report, "2021 iot security landscape @ONLINE," 2023.
- [4] M. Antonakakis, T. April, M. Bailey, M. Bernhard, E. Bursztein, J. Cochran, Z. Dumeric, J. A. Halderman, L. Invernizzi, M. Kallitsis *et al.*, "Understanding the mirai botnet," in *USENIX Security Symposium*, 2017.
- [5] N. Redini, A. Machiry, R. Wang, C. Spensky, A. Continella, Y. Shoshitaishvili, C. Kruegel, and G. Vigna, "Karonte: Detecting insecure multi-binary interactions in embedded firmware," in *2020 IEEE Symposium on Security and Privacy (SP)*. IEEE, 2020, pp. 1544–1561.
- [6] L. Chen, Y. Wang, Q. Cai, Y. Zhan, H. Hu, J. Linghu, Q. Hou, C. Zhang, H. Duan, and Z. Xue, "Sharing more and checking less: Leveraging common input keywords to detect bugs in embedded systems," in *30th USENIX Security Symposium (USENIX Security 21)*, 2021, pp. 303–319.
- [7] K. Cheng, Y. Zheng, T. Liu, L. Guan, P. Liu, H. Li, H. Zhu, K. Ye, and L. Sun, "Detecting vulnerabilities in Linux-based embedded firmware with SSE-based on-demand alias analysis," in *Proceedings of the 32nd ACM SIGSOFT International Symposium on Software Testing and Analysis*, 2023, pp. 360–372.
- [8] angr memember, "A reaching definition engine for binary analysis built-in in angr @ONLINE," https://degrigis.github.io/posts/angr_rd/, Dec. 2021.
- [9] K. Lu, A. Pakki, and Q. Wu, "Detecting {Missing-Check} bugs via semantic- and {Context-Aware} criticalness and constraints inferences," in *28th USENIX Security Symposium (USENIX Security 19)*, 2019, pp. 1769–1786.
- [10] W. Gibbs, A. S. Raj, J. M. Vadayath, H. J. Tay, J. Miller, A. Ajayan, Z. L. Basque, A. Dutcher, F. Dong, X. Maso *et al.*, "Operation mango: Scalable discovery of taint-style vulnerabilities in binary firmware services."
- [11] C. Heffner, "binwalk - firmware analysis tool designed to assist in the analysis, extraction, and reverse engineering of firmware images," <https://github.com/ReFirmLabs/binwalk>, Sep. 2023.
- [12] Y. Shoshitaishvili, R. Wang, C. Salls, N. Stephens, M. Polino, A. Dutcher, J. Grosen, S. Feng, C. Hauser, C. Kruegel, and G. Vigna, "SoK: (State of) The Art of War: Offensive Techniques in Binary Analysis," in *IEEE Symposium on Security and Privacy*, 2016.
- [13] Xavier Maso, "Handle function calls during static analysis in angr @ONLINE," <https://blog.xaviermaso.com/2021/02/25/Handle-function-calls-during-static-analysis-with-angr.html>, Feb. 2023.
- [14] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, *Introduction to algorithms*. MIT press, 2022.
- [15] Y. Wu, J. Wang, Y. Wang, S. Zhai, Z. Li, Y. He, K. Sun, Q. Li, and N. Zhang, "Your firmware has arrived: A study of firmware update vulnerabilities," in *USENIX Security Symposium*, 2024.
- [16] Y. Wu, Y. Wang, S. Zhai, Z. Li, A. Li, J. Wang, and N. Zhang, "Work-in-progress: Measuring security protection in real-time embedded firmware," in *IEEE Real-Time Systems Symposium (RTSS)*, 2022.
- [17] Windriver, "Windriver. windriver," <https://www.windriver.com/>, 2024.
- [18] W. Zhu, R. Liu, J. Wang, and Y. Zhou, "Vxhunter: A tool set for vxworks based embedded device analyses," in *Smart Grid/Industrial Security, ICML 2019, BAY SANDS, Singapore, March 26-29, 2019*. Blackhat-asia, 2019. [Online]. Available: <https://i.blackhat.com/asia-19/Fri-March-29/bh-asia-Zhu-Dive-into-VxWorks-Based-IoT-Device-Debug-the-Undebugable-Device.pdf>
- [19] A. A. Clements, L. Carpenter, W. A. Moeglein, and C. Wright, "Is your firmware real or re-hosted? a case study in re-hosting vxworks control system firmware," in *Proceedings of the 2021 NDSS Workshop on Binary Analysis Research (NDSS BAR 2021)*, 2021.
- [20] HexRays, "Hexrays IDA Pro," <https://www.hex-rays.com/products/ida/index.shtml> @ONLINE, Feb. 2023.
- [21] Y. Zheng, A. Davanian, H. Yin, C. Song, H. Zhu, and L. Sun, "FIRM-AFL: High-Throughput greybox fuzzing of IoT firmware via augmented process emulation," in *28th USENIX Security Symposium (USENIX Security 19)*, 2019, pp. 1099–1114.
- [22] B. Yu, P. Wang, T. Yue, and Y. Tang, "Poster: Fuzzing iot firmware via multi-stage message generation," in *Proceedings of the 2019 ACM SIGSAC conference on computer and communications security*, 2019, pp. 2525–2527.
- [23] A. Qasem, P. Shirani, M. Debbabi, L. Wang, B. Lebel, and B. L. Agba, "Automatic vulnerability detection in embedded devices and firmware: Survey and layered taxonomies," *ACM Computing Surveys (CSUR)*, vol. 54, no. 2, pp. 1–42, 2021.
- [24] Z. Gao, C. Zhang, H. Liu, W. Sun, Z. Tang, L. Jiang, J. Chen, and Y. Xie, "Faster and better: Detecting vulnerabilities in Linux-based IoT firmware with optimized reaching definition analysis."